

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL
ESCOLA POLITÉCNICA
CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE
COMPUTAÇÃO GRÁFICA

CAROLINA MICHEL FERREIRA

RELATÓRIO DO TRABALHO 2
TANQUE ARTICULADO ATIRADOR

https://youtu.be/yMhgV_Upghk

Porto Alegre

2023

INTRODUÇÃO

O presente relatório descreve o desenvolvimento de um projeto em OpenGL com o objetivo de criar um jogo em 3D, no qual o usuário assume o controle de um veículo em forma de tanque articulado. A proposta do jogo é proporcionar uma experiência interativa na qual o usuário deve destruir inimigos posicionados atrás de um paredão, utilizando um canhão articulado para lançar projéteis. O cenário é composto por um piso e um paredão, modelados em polígonos quadrados de dimensões específicas e texturizados para proporcionar uma ambientação visual realista.

Além disso, o veículo em forma de tanque apresenta articulações controláveis pelo usuário, permitindo movimentação no plano e ajustes no canhão para melhor direcionamento dos projéteis. O lançamento dos projéteis utiliza uma curva *Bézier* para determinar a trajetória. O jogo também incorpora objetos amigos e inimigos em forma de cachorros, adicionando desafios e estratégias ao gameplay.

O relatório abordará aspectos essenciais do desenvolvimento, desde a modelagem e texturização do cenário até a implementação das interações do veículo e dos projéteis. A detecção de colisões, a reconfiguração dinâmica do paredão após o impacto dos projéteis e a exibição correta dos objetos serão elementos centrais discutidos ao longo do documento.

DESENVOLVIMENTO

1. Desenho do Paredão

O método *DrawWall()* é a função que criei para desenhar o paredão. Ele gerencia a renderização dos ladrilhos individuais que compõem a parede, definindo a textura ou a cor da parede (dependendo da variável booleana *showTexture()*). Em seguida, a função aplica transformações de translação e rotação para posicionar e orientar a parede no ambiente gráfico. Utilizando loops for, percorro uma matriz booleana (representando os ladrilhos da parede) e desenho cada bloco utilizando a função *DrawTile()* sempre que um bloco estiver presente na matriz. Finalmente, a função restaura o estado anterior das transformações para manter a consistência na cena gráfica, possibilitando a continuidade do desenho de outros elementos após o término da renderização da parede.

2. Detecção da colisão do projétil

O método *DrawProjectile()* é responsável por controlar o movimento e detectar colisões do projétil no ambiente do jogo. Ele começa verificando se o jogo acabou por meio da função *handleGameOver()*. Se o jogo tiver terminado, a função encerra imediatamente. Em seguida, o código define um valor de tempo *DeltaT* para saber quantos pontos irão ter ao redor da curva *Bèzier*. O loop *while* é responsável por calcular a posição do projétil ao longo da trajetória definida pela função de *Bèzier*, representada por *CalculateBezier3()*. Dentro deste loop, várias condições são verificadas para detectar colisões do projétil com diferentes elementos do jogo. *HitVehicle()*, *HitWall()*, *HitFloor()* e *HitFrenemy()* são funções chamadas para verificar se o projétil colidiu com o veículo, com a parede, com o chão ou com um personagem "frenemy" (um tipo criado que contém as características dos personagens amigos e inimigos). Cada uma destas funções verifica se a posição atual do projétil, colide com o objeto correspondente no jogo. As colisões são detectadas quando:

- Se o projétil atingir um veículo, é exibida uma mensagem indicando a colisão, a pontuação é ajustada e o loop é interrompido.
- Se o projétil atingir uma parede, a mensagem de colisão é exibida, a parede é quebrada, a pontuação é atualizada e o loop é interrompido.
- Se o projétil atingir o chão, uma mensagem é exibida indicando a colisão, a pontuação é ajustada e o loop é interrompido.

- Se o projétil atingir um "frenemy", é exibida uma mensagem indicando se foi um amigo ou inimigo, a pontuação é ajustada de acordo e o número de "frenemies" é atualizado. Se não houver mais "frenemies" no jogo, o jogo é encerrado.

O loop continua até que o projétil atinja o final de sua trajetória ou uma colisão seja detectada, interrompendo o loop e o processo de movimento e colisão do projétil.

3. Reconfiguração do paredão após a colisão com o projétil

Quando há uma detecção de colisão entre o projétil e a parede, o procedimento é iniciar a reconfiguração da estrutura da parede. Após identificar o ponto de colisão, obtenho as suas coordenadas X e Y, utilizando a função *round()* para determinar qual é o ladrilho central atingido. Em seguida, acesso a matriz booleana mencionada na seção “Desenho no Paredão”, marcando esse ladrilho como destruído para que não seja desenhado no próximo quadro do jogo. A etapa seguinte envolve a aplicação de operações matemáticas simples e verificações de limites. Para cada um dos 8 ladrilhos vizinhos ao ladrilho central atingido, verifico se estão dentro dos limites de desenho. Isso é feito para evitar a destruição de todos os 8 ladrilhos e, em vez disso, destruir apenas aqueles adjacentes ao ladrilho central. Através do aumento ou diminuição das coordenadas X e Y em relação ao ponto central, marco também esses ladrilhos como destruídos na grade, completando assim a atualização da parede após a colisão com o projétil.

4. Modelagem do veículo

O método *DrawVehicle()* é responsável por renderizar o, dividindo a sua representação em duas partes principais: a base e o canhão. Estas partes são desenhadas como paralelepípedos, podem possuir textura e suas posições e direções são definidas por variáveis. Em resumo, o método utiliza transformações de translação e rotação para posicionar e orientar a base e o canhão do veículo, desenhando-os como paralelepípedos, com opções para texturização ou cores sólidas, dependendo da variável *showTexture*.

5. Movimentação do veículo

Para permitir o movimento de giro do veículo, desenvolvi o método *RotateVehicle()* que permite ao jogador girar o veículo para a esquerda (tecla 'a') ou para a direita (tecla 'A'). Quando uma dessas teclas é pressionada, o ângulo de rotação do veículo é ajustado. A rotação é realizada adicionando ou subtraindo 1 grau ao ângulo de rotação do veículo

dependendo da tecla pressionada. Limites são aplicados para garantir que o ângulo permaneça dentro do intervalo de -360 a 360 graus, evitando valores extremos.

Já para movimentar o veículo para frente e para trás, criei o método *MoveVehicle()* que movimenta o veículo conforme a tecla pressionada ('W' ou 'S', respectivamente). Tenho um vetor que representa a frente do veículo com base no ângulo atual de orientação do veículo. A distância que o veículo avança é determinada multiplicando a frente do veículo pela distância de movimento percorrido. A direção é invertida se a tecla 'S' ou 's' for pressionada, permitindo ao veículo mover-se para trás. A nova posição do veículo é calculada adicionando a posição atual com a distância percorrida. São realizadas várias verificações para garantir que o veículo não ultrapasse os limites da pista ou colida com paredes no método *VehicleCanMove()*.

O movimento é baseado na orientação do veículo e na distância de deslocamento em direções específicas, enquanto verificações são aplicadas para garantir a validade e a integridade do movimento dentro da cena.

6. Lançamento do projétil

No método *ShootProjectile()* defino a direção do canhão e o cálculo da trajetória. Este método define a direção inicial do canhão como um vetor apontando para a frente, rotaciona o vetor com base no ângulo atual do canhão, calcula a posição inicial do canhão e determina a direção de lançamento do projétil somando a posição relativa do canhão com a direção calculada multiplicada pela força do canhão. Além disso, o método calcula um alvo para o projétil a partir da posição atual do canhão e do vetor z (direção para a frente) e define os pontos de controle para a curva *Bézier* que calcula a trajetória. Já o método *DrawProjectile()* desenha o projétil e calcula a posição do projétil na curva *Bézier*, verificando se houveram colisões com os elementos do jogo.

Estes métodos trabalham juntos para calcular a trajetória do projétil com base na orientação do veículo e no ângulo do canhão, desenhando-o enquanto se move ao longo da curva de *Bézier*, verificando colisões e interagindo com elementos do jogo.

CONCLUSÃO

O desenvolvimento do projeto em OpenGL para criar um jogo tridimensional com um veículo robô articulado proporcionou uma experiência desafiadora e enriquecedora. A implementação de elementos como o cenário, o veículo, os projetos e os objetos adicionaram complexidade ao código, exigindo a aplicação de conceitos fundamentais de computação gráfica.

A modelagem e texturização do cenário, com destaque para o piso e o paredão, visaram criar um ambiente visualmente envolvente, contribuindo para a interação do usuário no jogo. A utilização de texturas específicas para cada elemento proporcionou uma representação gráfica consistente e esteticamente agradável.

A dinâmica do veículo, com suas articulações controláveis, permite ao usuário explorar o cenário de forma interativa, adicionando um aspecto de jogabilidade mais rico. A implementação do lançamento de projetos, com a consideração das leis da física ou curvas de *Bézier*, contribuiu para a realidade e o desafio do jogo.

A detecção de colisões e a reconfiguração dinâmica do paredão após impactos revelou a aplicação efetiva de algoritmos para jogo lidar com interações complexas entre elementos do jogo. A inclusão de objetos amigos e inimigos proporcionou uma experiência mais completa e estratégica, enriquecendo a jogabilidade.

Em resumo, a conclusão deste projeto não apenas atendeu aos requisitos propostos, mas também proporcionou uma oportunidade potencial de aplicar conhecimentos teóricos em um contexto prático e desafio. O resultado reflete a habilidade em integrar conceitos de computação gráfica, física simulada e lógica de jogo, culminando em uma experiência interativa e visualmente atraente.